

Tema 0.

Repaso de Python para lingüistas

Primer encargo en LexiData: preparar textos reales para análisis lingüístico computacional

Material teórico | Programación para PLN | Universidad Pontificia Comillas

Título narrativo

Primer encargo en LexiData: preparar textos reales para análisis lingüístico computacional

Contexto profesional

Imagina que acabas de incorporarte como estudiante en prácticas a **LexiData**, una pequeña consultora de lingüística computacional. El equipo trabaja con universidades, editoriales, medios de comunicación y departamentos de comunicación corporativa. Su labor consiste en transformar textos humanos en datos que puedan analizarse: contar palabras, detectar patrones, limpiar errores, comparar versiones, preparar corpus y construir pequeños prototipos de PLN.

Tu primer encargo no es crear una inteligencia artificial compleja. Es algo más importante: aprender a manejar Python como herramienta de trabajo lingüístico. Antes de analizar millones de palabras, hay que saber representar un texto, guardarlo en variables, dividirlo en tokens, recorrer colecciones, tomar decisiones con condicionales y escribir pequeñas funciones reutilizables.

1. Qué es Python y por qué le interesa a una lingüista

Python es un lenguaje de programación. Un lenguaje de programación permite dar instrucciones a un ordenador de forma precisa. En lingüística computacional usamos Python porque:

- es legible y relativamente cercano al lenguaje humano;
- funciona en distintos sistemas operativos;
- tiene muchas bibliotecas para trabajar con texto;
- se usa mucho en PLN, ciencia de datos, investigación y docencia;
- permite pasar de tareas pequeñas, como contar palabras, a proyectos más grandes, como clasificar documentos.

Para una lingüista, Python no sustituye el análisis lingüístico. Lo amplía. Permite comprobar hipótesis sobre textos, automatizar tareas repetitivas y trabajar con corpus que serían imposibles de revisar manualmente.

Ejemplo profesional:

```
texto = "La lengua cambia cuando cambia la comunidad que la usa."  
print(texto)
```

Aquí Python guarda una frase y la muestra en pantalla. Parece sencillo, pero es el primer paso para cualquier tarea de PLN.

2. Scripts y notebooks

En Python podemos trabajar principalmente de dos formas.

2.1. Scripts

Un **script** o **guion** es un archivo de texto con extensión `.py`. Contiene instrucciones Python que se ejecutan normalmente de principio a fin.

Ejemplo:

```
print("Análisis de corpus iniciado")
```

En un entorno profesional, los scripts se usan cuando queremos automatizar una tarea que se repetirá muchas veces: limpiar cien archivos, contar frecuencias en un corpus o generar informes.

2.2. Notebooks

Un **notebook** permite combinar celdas de código y celdas de texto. Es muy útil para aprender, experimentar y explicar un proceso paso a paso.

En un notebook podemos escribir una instrucción:

```
1 + 1
```

Y ver inmediatamente el resultado.

También podemos mezclar código con explicaciones teóricas. Por eso los notebooks son frecuentes en docencia, investigación exploratoria y prototipado.

Atajos habituales:

- `Shift + Enter`: ejecuta la celda y pasa a la siguiente.
- `Ctrl + Enter`: ejecuta la celda sin moverse.
- `Alt + Enter`: ejecuta la celda y crea una nueva debajo.

3. Expresiones, comentarios y salida por pantalla

Una **expresión** es una instrucción que Python puede evaluar para obtener un resultado.

```
1
```

```
1 + 1
```

Los comentarios empiezan con `#`. Python los ignora, pero ayudan a que una persona entienda el código.

```
# Esto es un comentario
# Sirve para explicar que hace el código
print(3 - 2)
```

La función `print()` muestra información en pantalla:

```
print("Hola, equipo de LexiData")
```

En notebooks hay una diferencia importante:

```
"Hola"
```

devuelve la representación del objeto, mientras que:

```
print("Hola")
```

muestra el texto como queremos leerlo.

4. Operadores básicos

Python puede funcionar como una calculadora. Los operadores más básicos son:

Operador	Significado	Ejemplo	Resultado
+	suma	2 + 3	5
-	resta	5 - 2	3
*	multiplicación	2 * 3	6
/	división decimal	4 / 3	1.333...
//	división entera	4 // 3	1
%	resto o módulo	16 % 12	4
**	potencia	2 ** 3	8

El operador % es muy útil para saber si un número es par:

```
n = 8
n % 2 == 0
```

Si el resto de dividir entre 2 es 0, el número es par.

5. Strings: representar texto en Python

En PLN casi todo empieza con texto. En Python, el texto se representa mediante **strings** o cadenas de caracteres.

```
"Hola mundo"
```

```
'Hola mundo'
```

Podemos usar comillas dobles o simples. Lo importante es abrir y cerrar con el mismo tipo.

5.1. Comillas dentro de strings

Si queremos incluir comillas dentro de un texto, podemos escapar las comillas internas con \.

```
frase = "El titular decía que \"la lengua también innova\"."
print(frase)
```

Escapar un carácter significa quitarle su función especial para que Python lo interprete como parte del texto.

5.2. Saltos de línea y tabulaciones

Algunos caracteres no se escriben directamente dentro de una línea. Se representan con secuencias de escape:

- \n: salto de línea.
- \t: tabulación.

```
print("palabra\tcategoria\ncasa\tsustantivo\ncorrer\tverbo")
```

Esto produce una salida parecida a una pequeña tabla.

5.3. Comillas triples

Cuando queremos escribir textos largos con varios renglones, usamos comillas triples:

```
texto = """Este es el primer párrafo.  
Este es el segundo.  
Este es el tercero."""  
  
print(texto)
```

Este recurso es muy útil para guardar fragmentos de corpus, enunciados, transcripciones o ejemplos largos.

6. Variables y objetos

Una **variable** es un nombre que usamos para guardar un objeto.

```
texto = "Barriga llena, corazón contento"
```

Ahora no tenemos que repetir la frase completa. Podemos usar el nombre `texto`.

```
print(texto)
```

En Python, casi todo es un **objeto**: números, strings, funciones, listas, diccionarios. Un objeto es una entidad que Python puede manipular.

6.1. Nombres de variables

Buenas prácticas:

- usar nombres claros;
- evitar espacios;
- usar guiones bajos si hace falta;
- evitar tildes y caracteres problematicos;
- elegir nombres relacionados con el contenido.

Mejor:

```
frase_original = "La lengua cambia."
```

Peor:

```
x = "La lengua cambia."
```

x funciona, pero no explica nada. En proyectos lingüísticos, la claridad importa.

6.2. Identidad y valor: `is` frente a `==`

Dos variables pueden apuntar al mismo objeto:

```
cadena = "manzana\ntpera\nnaranja\tplatano"
otro_nombre = cadena
cadena is otro_nombre
```

`is` pregunta si dos variables se refieren exactamente al mismo objeto.

En cambio, `==` pregunta si dos objetos tienen el mismo valor:

```
cadena2 = "manzana\ntpera\nnaranja\tplatano"
cadena == cadena2
```

Para comparar contenido textual normalmente usamos `==`, no `is`.

7. Atributos y métodos

Muchos objetos tienen información asociada y acciones disponibles.

Un **atributo** es información asociada a un objeto.

```
c = 1 + 2j
c.real
c.imag
```

Un **método** es una función asociada a un objeto. Se llama con paréntesis.

```
c.conjugate()
```

En strings, los métodos son especialmente importantes:

```
"gato".islower()
"PERRO".isupper()
"Maria".istitle()
"5543".isnumeric()
```

Estos métodos devuelven `True` o `False`.

7.1. Reemplazar texto

El método `.replace()` sustituye una parte del string por otra:

```
frase = "Me gusta analizar poesia y teatro."
frase.replace("poesia", "narrativa")
```

Importante: los strings son **inmutables**. Esto significa que no se modifican en el sitio. El método crea un string nuevo.

```
animal = "ornitorrinco"
animal.upper()
animal
```

`animal` sigue teniendo el valor original. Si queremos guardar el cambio:

```
animal = animal.upper()
```

Esta idea es clave en limpieza de texto: cada transformación suele producir una nueva versión del texto.

8. Colecciones: agrupar datos lingüísticos

Una colección permite guardar varios elementos juntos. En lingüística computacional se usan para representar tokens, vocabularios, pares de traducción, frecuencias, etiquetas gramaticales y muchos otros datos.

8.1. Listas

Una **lista** se escribe entre corchetes.

```
tokens = ["Barriga", "llena", ",", "corazon", "contento"]
```

Las listas tienen orden y pueden modificarse.

```
tokens[0]
```

devuelve el primer elemento, porque en Python se empieza a contar desde 0.

8.2. Tuplas

Una **tupla** se escribe entre paréntesis.

```
etiqueta = ("casa", "sustantivo")
```

Se parece a una lista, pero no se puede modificar después de crearla. Es útil cuando queremos representar un par estable, por ejemplo palabra y categoría.

8.3. Diccionarios

Un **diccionario** guarda pares clave-valor.

```
traducciones = {"blue": "azul", "red": "rojo"}
```

Para acceder a un valor:

```
traducciones["blue"]
```

En PLN, los diccionarios son esenciales para frecuencias:

```
frecuencias = {"lengua": 3, "texto": 5}
```

La clave es la palabra. El valor es el número de apariciones.

Las claves deben ser únicas. Si repetimos una clave, Python conserva la última versión.

```
{"impar": 1, "par": 2, "impar": 3}
```

8.4. Conjuntos

Un **conjunto** almacena valores únicos.

```
vocabulario = {"los", "trapecistas", "son", "los", "mejores"}
```

El repetido desaparece. Por eso los conjuntos son muy útiles para obtener vocabulario único de un texto.

El conjunto vacío se crea con:

```
set()
```

No con {}, porque {} crea un diccionario vacío.

9. Funciones comunes para colecciones

9.1. `len()`

`len()` devuelve la longitud de una colección.

```
len(["pera", "manzana", "naranja"])
```

En una lista cuenta elementos. En un string cuenta caracteres.

```
len("Reino de Espana")
```

En un diccionario cuenta entradas.

```
len({"naranja": "orange", "manzana": "apple"})
```

9.2. `in`

`in` pregunta si un elemento pertenece a una colección.

```
"azul" in ["rojo", "amarillo", "verde", "azul"]
```

En diccionarios, `in` busca en las claves, no en los valores:

```
"manzana" in {"manzana": "apple", "naranja": "orange"}
```

```
"apple" in {"manzana": "apple", "naranja": "orange"}
```

La segunda expresión devuelve `False` porque "apple" es valor, no clave.

10. Índices y slicing

Los strings, listas y tuplas tienen orden. Podemos acceder a sus elementos por posición.

```
palabra = "comadreja"  
palabra[0]
```

devuelve "c".

Python cuenta desde 0:

Indice	Caracter
0	c
1	o
2	m
3	a
4	d

10.1. Slicing

El slicing permite extraer una parte:

```
palabra[1:4]
```

devuelve desde la posición 1 hasta la anterior a la 4.

```
palabra[1:]
```

desde la posición 1 hasta el final.

```
palabra[:4]
```

desde el inicio hasta la posición anterior a 4.

```
palabra[-3:]
```

los tres últimos caracteres.

Esto es útil para estudiar terminaciones, prefijos, sufijos o fragmentos de palabras.

No podemos usar índices directamente en diccionarios ni conjuntos:

```
dic = {"naranja": "orange", "manzana": "apple"}  
# dic[0] no funciona
```

Un diccionario se consulta por clave, no por posición.

11. Eliminar elementos

del elimina elementos en colecciones mutables, como listas y diccionarios.

```
lista = ["gato", "perro", "pato"]  
del lista[0]  
lista
```

En diccionarios eliminamos por clave:

```
dic = {"casa": "house", "ventana": "window"}  
del dic["casa"]  
dic
```

No podemos eliminar una entrada de diccionario indicando el valor:

```
# del dic["house"] no funciona si "house" no es clave
```

Las tuplas no permiten eliminar elementos por posición porque son inmutables.

Los conjuntos no se consultan por índice porque no tienen orden posicional.

12. Control de flujo: tomar decisiones

Hasta ahora el código se ejecutaba de arriba abajo. Pero muchas tareas requieren decisiones.

12.1. Condicionales `if` y `else`

```
n = 5  
  
if n % 2 == 0:  
    print("par")  
else:  
    print("impar")
```

La estructura significa:

- si la condición es verdadera, ejecuta el bloque del `if`;
- si no, ejecuta el bloque del `else`.

La indentación es obligatoria. Python usa los espacios para saber que instrucciones pertenecen a cada bloque.

Aplicación lingüística:

```
palabra = "casa"  
  
if len(palabra) > 5:  
    print("palabra larga")  
else:  
    print("palabra corta")
```

12.2. `if` sin `else`

No siempre necesitamos alternativa.

```
palabra = "comunicación"

if len(palabra) > 10:
    print("Posible palabra larga para revisar")

print("Análisis terminado")
```

13. Bucles: repetir una tarea

Un bucle `for` permite recorrer una colección elemento por elemento.

```
palabras = ["Barriga", "llena", "corazon", "contento"]

for palabra in palabras:
    print(palabra)
```

En PLN esto es fundamental. Normalmente no analizamos una sola palabra, sino muchas.

Ejemplo: calcular la longitud media de las palabras de una frase.

```
frase = "Barriga llena corazon contento"
palabras = frase.split(" ")

suma_longitudes = 0

for palabra in palabras:
    suma_longitudes = suma_longitudes + len(palabra)

media = suma_longitudes / len(palabras)
print(media)
```

`.split(" ")` divide el string usando el espacio como separador. Es una forma básica de tokenización.

14. Funciones: crear herramientas reutilizables

Una función es un bloque de código que recibe datos, realiza una tarea y puede devolver un resultado.

```
def nombre_funcion(entrada):
    # instrucciones
    return resultado
```

Ejemplo profesional: contar frecuencias de palabras.

```
def frecuencia_texto(texto):
    frecuencias = {}

    for palabra in texto.split(" "):
        if palabra not in frecuencias:
            frecuencias[palabra] = 1
        else:
            frecuencias[palabra] = frecuencias[palabra] + 1

    return frecuencias
```

Uso:

```
frecuencia_texto("Barriga llena corazon contento")
```

Esta función:

1. crea un diccionario vacío;
2. divide el texto en palabras;
3. recorre cada palabra;
4. si la palabra no estaba, la añade con frecuencia 1;
5. si ya estaba, suma 1;
6. devuelve el diccionario final.

15. Limpieza básica de texto

En textos reales aparecen mayúsculas, signos de puntuación y variantes que pueden dificultar el conteo.

Si contamos directamente:

```
"Barriga" != "barriga"  
"contento" != "contento."
```

Para Python son strings distintos. Por eso limpiamos el texto.

15.1. Limpieza manual

```
def frecuencia_texto_limpiar(texto):  
    signos = [".", ",", ";", "...", "¿", "?", "¡", "!"]  
  
    texto = texto.lower()  
  
    for signo in signos:  
        texto = texto.replace(signo, "")  
  
    frecuencias = {}  
  
    for palabra in texto.split():  
        if palabra not in frecuencias:  
            frecuencias[palabra] = 1  
        else:  
            frecuencias[palabra] += 1  
  
    return frecuencias
```

`lower()` pasa el texto a minúsculas.

`replace()` elimina signos.

`split()` divide por espacios en blanco.

15.2. Limpieza con `string`

Python tiene módulos que amplían sus capacidades. El módulo `string` incluye una lista de signos de puntuación frecuentes.

```
import string

def frecuencia_texto_limpia(texto):
    texto = texto.lower()
    texto = texto.translate(str.maketrans("", "", string.punctuation))

    frecuencias = {}

    for palabra in texto.split():
        if palabra not in frecuencias:
            frecuencias[palabra] = 1
        else:
            frecuencias[palabra] += 1

    return frecuencias
```

Esta versión es más compacta, aunque conviene recordar que `string.punctuation` está pensado sobre todo para puntuación ASCII. En textos españoles puede que algunos signos o caracteres requieran tratamiento adicional.

16. Importar librerías: NLTK y tokenización

Una librería es un conjunto de código ya preparado que podemos reutilizar.

`nltk` significa **Natural Language Toolkit**. Es una biblioteca clásica de PLN en Python. Permite trabajar con tokenización, corpus, etiquetado, clasificación y otras tareas.

Para usar una librería:

```
import nltk
```

Para importar una herramienta concreta:

```
from nltk.tokenize import TreebankWordTokenizer
```

Después creamos un tokenizador:

```
tokenizer = TreebankWordTokenizer()
```

Y lo aplicamos:

```
frase = "Barriga llena, corazon contento."
tokens = tokenizer.tokenize(frase)
print(tokens)
```

La tokenización divide un texto en unidades llamadas **tokens**. Un token puede ser una palabra, un signo de puntuación o un número.

Comparación:

```
frase.split(" ")
```

divide solo por espacios.

```
tokenizer.tokenize(frase)
```

aplica reglas más sofisticadas y puede separar signos de puntuación.

Para una lingüista computacional, tokenizar es uno de los primeros pasos antes de contar, clasificar, etiquetar o modelar un texto.

17. Errores habituales y como interpretarlos

17.1. `SyntaxError`

Ocurre cuando Python no entiende la estructura del código.

Ejemplo:

```
"El informe dice que "el corpus está limpio""
```

Python cree que el string termina antes de tiempo. Solución:

```
"El informe dice que \"el corpus está limpio\""
```

17.2. Error por índice

Si pedimos una posición que no existe:

```
palabra = "casa"  
palabra[10]
```

Python no puede devolver el carácter 10 porque la palabra solo tiene cuatro caracteres.

17.3. Error por clave

```
dic = {"casa": "house"}  
dic["house"]
```

No funciona porque "house" es valor, no clave.

17.4. Error por olvidar paréntesis en métodos

```
texto = "hola"  
texto.upper
```

Esto muestra el método, pero no lo ejecuta. Para ejecutarlo:

```
texto.upper()
```

18. Cierre narrativo

Al terminar estos ejercicios, no solo habrás aprendido una lista de comandos. Habrás construido el primer prototipo de una tarea real de lingüística computacional: transformar comentarios escritos en datos analizables.

Ese es el cambio central del curso: pasar de leer textos uno a uno a diseñar procedimientos que permitan explorarlos de forma sistemática. Python no piensa por la lingüista, pero le permite observar patrones que antes quedaban ocultos por el volumen de datos.